

Introduction Data Warehouse and Snowflake



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://about.softuni.bg>

1. Data Warehouse
2. Data Warehouse Architecture
3. Snowflake
 - Overview
 - Architecture
 - Pricing
 - Layers, Operations, and Access Controls
 - Use Case





Data Warehouse

Purposes, Concepts and Architecture

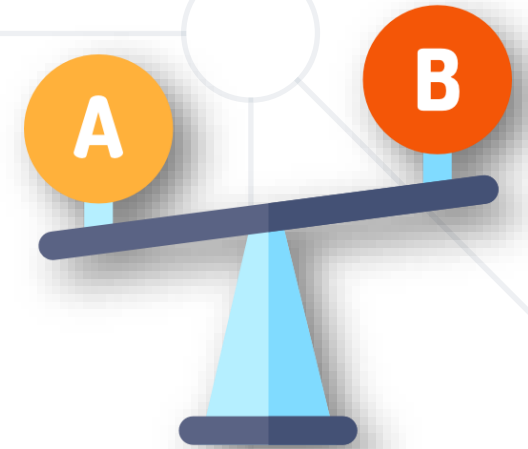
- What is **Data Warehouse**?
 - A Data Warehouse (DW) is a **centralized storage system** that **collects, stores, and processes** structured data from multiple sources for analysis, reporting, and decision-making



- Definition - Centralized Repository for Structured Data
- Purposes
 - A **large-scale repository** that integrates data from different systems
 - Optimized for **fast querying and analysis**, not for transaction processing
 - Designed for **historical data storage and trend analysis**
 - Executives to **analyze monthly sales trends**
 - Inventory managers to **predict stock shortages**

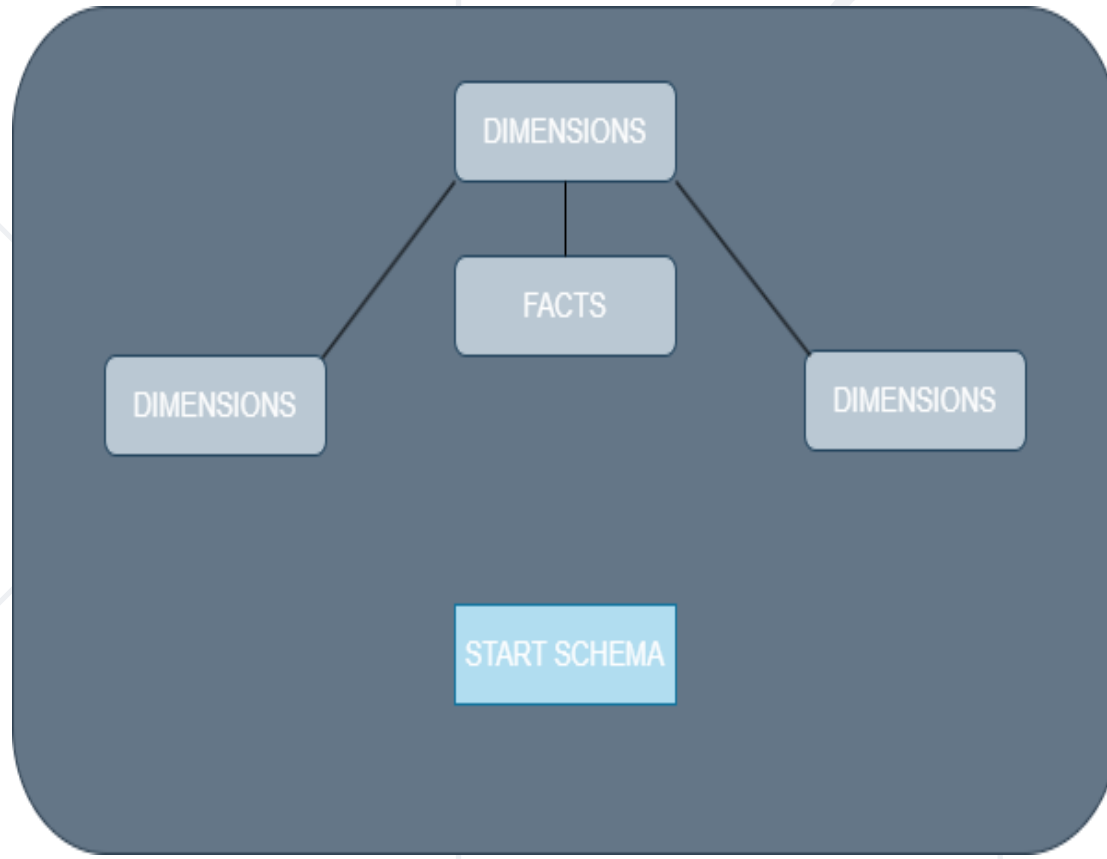
Difference Between OLTP and OLAP Systems

- Difference
 - **OLTP**: Transactional frequent updates, detailed data supports day-to-day operations (e.g., bank transactions)
 - **OLAP**: Analytical, read heavy, aggregated data, supports decision-making (e.g., sales trends)



- Dimensional modeling consists of two core table types -
Dimensional modeling is the fundamental technique for designing a data warehouse (DW). **Optimizes databases** for querying and reporting
 - Fact Tables - A **fact table** contains **quantitative (numeric) data** representing business transactions or events. Each record in a fact table is a **business event**, and it is linked to dimension tables via **foreign keys**
 - Dimension Tables - A **dimension table** stores **descriptive attributes** related to business entities (e.g., customers, products, time)

■ Star Schema



Dim_Product

product_id
name
category
subcategory
dimensions

Dim_Customer

customer_id
first_name
last_name
sex
city

Sales

sales_id
product_id
customer_id
units
price

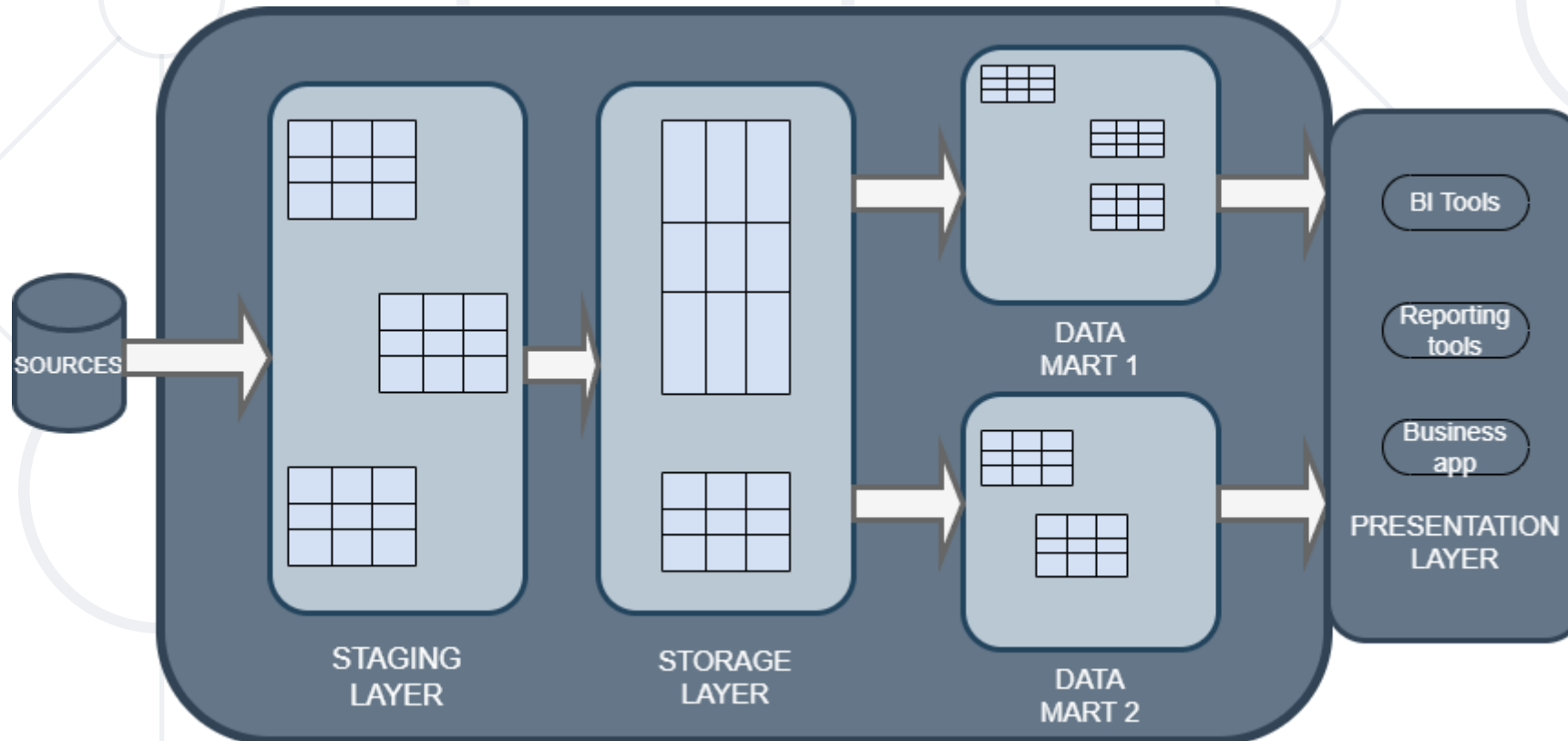
Dim_Date

date_id
year
quarter
month
day
weekday

- **Data Lake vs Data Warehouse**
 - They are used as **CENTRALIZED DATA STORAGE**
 - Key Differences

Aspect	Data Lake	Data Warehouse
Data Type	Raw (Structured, Semi-Structured and Unstructured)	Processed and structured
Schema	Schema on-read	Schema on-write
Use Cases	Data Science, ML, big data	BI, reporting, analytics
Cost	Cheaper(scales easily)	Expensive(performance-focused)

- Data Warehouse Architecture
 - Data Warehouse Layers



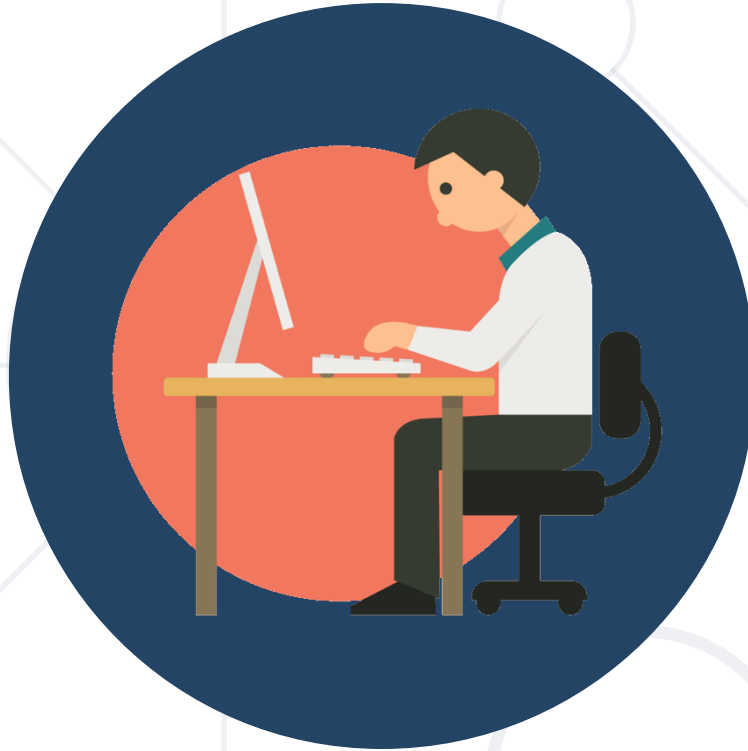
- **Staging Layer**
 - Landing zone extracted data
 - Data in tables and on a separate database
- **Storage Layer**
 - Inside the data warehouse keeps transformed data ready for analysis
- **Data Marts**
 - Data Marts is subset of the WH got the need of a specific department, business area or user group
- **Presentation Layer**
 - That's where user can access and analyze data from the storage layer

- **Traditional databases:** Designed for transactional workloads (OLTP)
- **Data warehouses:** Optimized for analytical workloads (OLAP)
- Limitations of traditional databases for analytical tasks
 - Slower performance for complex queries
 - Schema is not optimized for large-scale aggregations
 - Hard to scale horizontally for big data workloads

How Data Warehouses Optimize Analytics

- Data warehouses **solve** these issues with
 - **Columnar storage** for faster aggregations
 - **Denormalized schemas** (e.g., star schema) for simplified querying
 - **Built-in optimizations** for analytical queries

Feature	Traditional Database	Data Warehouse
Purpose	Operational(e.g., CRUD)	Analytical(e.g., BI, reports)
Schema Design	Normalized(minimize redundancy)	Denormalized(query optimization)
Data Volume	Handles smaller datasets	Optimize for massive datasets
Performance	Fast for transactions	Fast for complex queries
Concurrency	Few concurrent users (staff apps)	Many concurrent users
Examples	PostgreSQL, MySQL	Snowflake, Redshift, BigQuery



Practice

Live Exercise – Data Warehouse



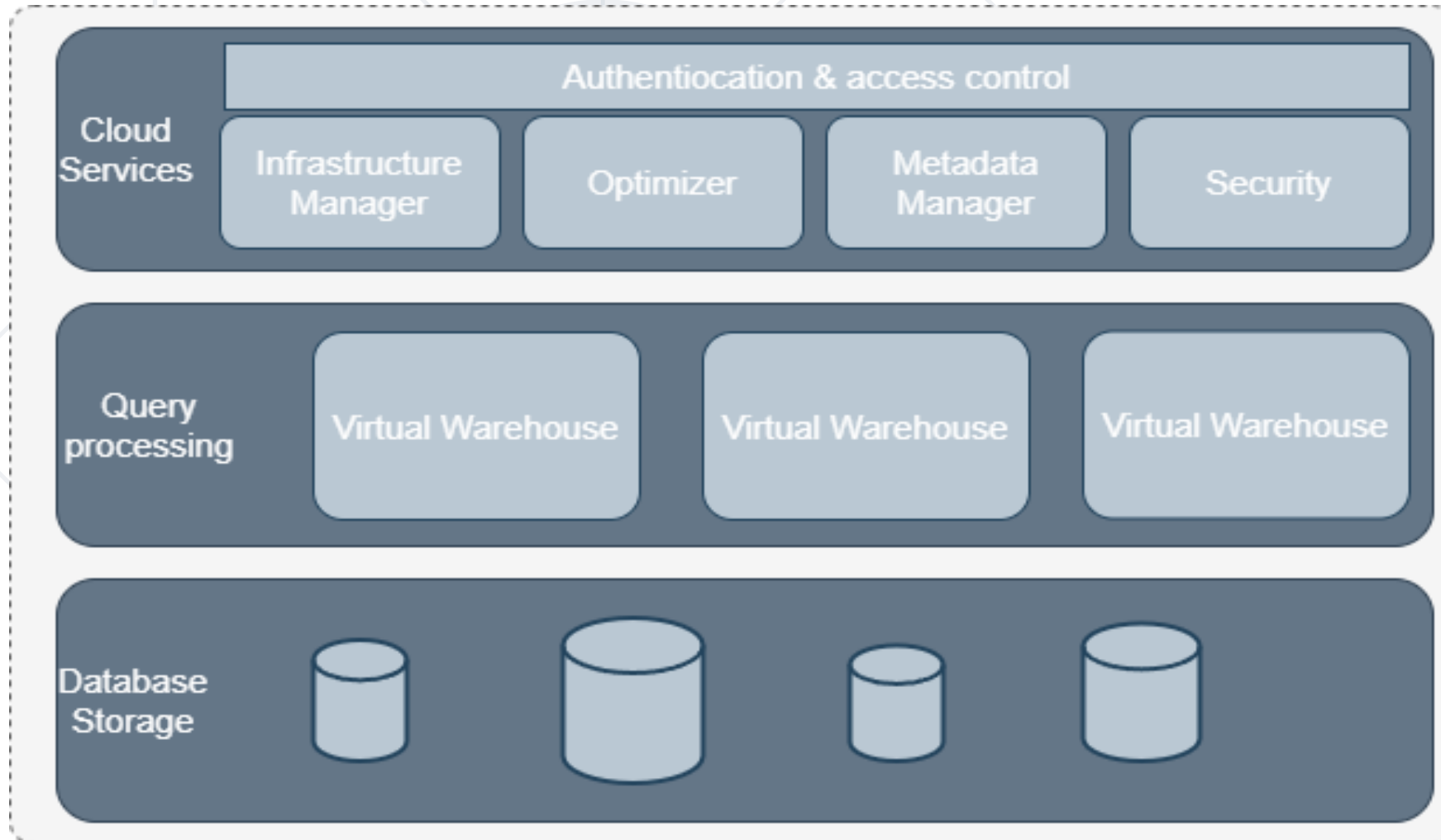
Snowflake

What is Snowflake?

- What is **Snowflake**?
 - Snowflake is a **fully managed, multi-cloud data platform** (AWS, Azure, and Google Cloud) designed for **modern data warehousing, analytics, and secure data collaboration**, supporting both **structured** and **semi-structured** data



Snowflake Architecture



What is Separation of Storage and Compute?

- **Storage** Layer
 - Centralized, columnar storage
 - Stores raw & processed data
 - Automatically compressed, encrypted, and versioned
 - Scales independently
 - Handles structured & semi-structured formats
- **Compute** Layer
 - Virtual Warehouses = Clusters of compute resources
 - Can scale **horizontally** and **vertically**
 - Multiple warehouses can read the **same data concurrently**
 - Pay only for active compute time

What is Separation of Storage and Compute?

- **Cloud Services** Layer
 - Authentication
 - Query parsing and optimization
 - Metadata Management
 - Orchestration and scheduling



Multi-Cluster Shared Data Architecture

- All compute clusters access a single version of the data
- Each **virtual warehouse (cluster)** operates **independently**
- Prevents **resource contention** (slow queries, wait times)

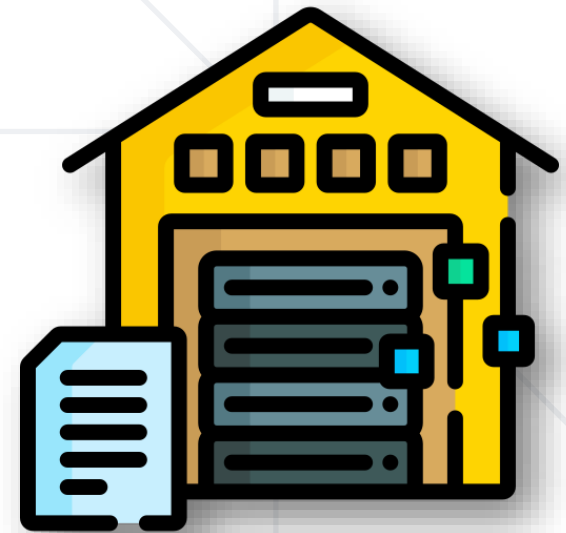


What is Separation of Storage and Compute

- In Snowflake, **Storage** and **Compute** work **independently**
- They **scale separately**, are **billed separately**, and **pause / resume separately**
- You can store **massive amounts of data** without needing to process it
- You can **run analytics only when needed** — saving cost

How Snowflake Differs from Trad. Warehouses

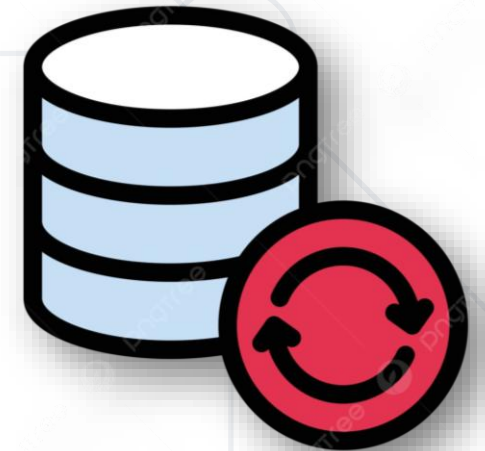
- Decoupling Compute and Storage
- Zero Copy Cloning
- Query Optimization with Caching



Compared to Redshift and BigQuery

Feature	Snowflake	Redshift	Big Query
Easy to Use	Fully managed, simple set up	Requires more tuning and management	Fully managed, serverless
Scaling	Auto-scales compute and storage independently	Compute and storage are tightly coupled	Auto-scales seamlessly
Semi-Structured Data	Native support for JSON, Parquet, and more	Limited support	Excellent support
Performance	Optimized for large-scale analytical workloads	Good, but may require manual tuning	Fast, especially for ad-hoc queries
Pricing Model	Pay-as-you-go, based on usage	Reserved instances or on-demand pricing	Pay-as-you-go, query-based pricing
Unique Features	Time travel, zero-copy cloning	Deep integration with AWS	Real-time streaming and ML integrations
Use Cases	Real-time analytics, multi-cloud environments	Large-scale analytics in AWS ecosystems	Real-time dashboards, AI/ML pipelines

- **Centralized, columnar storage**
- **Immutable** storage layers
- **Structured & Semi-Structured** (JSON, Avro, Parquet) data handling
- **Time Travel** and **Fail-safe**
- **Unstructured** data support (preview feature)



■ Compute

- Charged for active warehouses per hour
- Depending on the size of the warehouse
- Billed by second (minimum of 1min)
- Charged in Snowflake credits

■ Storage

- Monthly storage fees
- Based on average storage used per month
- Cost calculated after compression
- Cloud Providers

- Data Warehouse Layers in Snowflake
 - **Staging** Layer (Raw Ingest)
 - **Raw** Layer (Immutable Historical Store)
 - **Cleansed** Layer (Data Quality Checks)
 - **Business / Dimensional** Layer (Facts & Dimensions)
 - **Presentation** Layer (BI-ready)

- Snowflake tables types
 - A fully persistent table — data is stored **permanently** in Snowflake with **time travel** and **fail-safe protection Raw Layer** (Immutable Historical Store)
 - A **semi-permanent** table — physically stored but **no fail-safe**. Data can be recreated, so it's cheaper and faster **Business / Dimensional Layer** (Facts & Dimensions)
 - A table that exists **only during your session** (disappears when you log out).

- **Monitoring & Administration**
 - Resource Monitors (Cost Controls)
 - Account Usage Views
 - Information Schema Queries
 - Query Profiling Tools
 - Storage Usage Dashboards
 - Usage-Based Billing Insights

- Security and Governance
 - RBAC (Roles, Grants, Privileges)
 - **Role-Based Access Control (RBAC)** lets you assign permissions via **roles**
 - Dynamic Data Masking
 - Hides sensitive data from unauthorized users **in real time**
 - Row Access Policies
 - Filters **rows** returned based on user role or attributes
 - Column-Level Security
 - Restricts access to **specific columns** in a table
 - Data Classification (Sensitive Data Detection)
 - Helps identify **personally identifiable information (PII)** automatically

- Standard **SQL Syntax** (e.g SELECT, INSERT, JOIN, GROUP BY)
- Snowflake Specific SQL Functions are **unique SQL extensions** and **functions** optimized for cloud-based data warehousing

- Semi – Structured Data Handling (JSON)

```
SELECT json_data:key::string AS key_value FROM raw_table;
```

- Time Travel

```
SELECT * FROM my_table AT TIME AS OF '2023-01-01 00:00:00';
```

- Cloning

```
CREATE OR REPLACE CLONE my_table_clone FROM my_table
```

- **Common Scenarios**

- Real Time Analytics
- Data Lake Consolidation
- Machine Learning Pipelines

- **Industry Applications**

- **Retail** - Analyze sales trends across 100 stores to improve pricing strategies
- **Finance** - Detect anomalies in transactional data for fraud detection using historical data
- **Healthcare** - Aggregate patient data across clinics to provide centralized analytics for improving care outcomes

■ Retail

- **Use Case:** Snowflake's ability to scale resources dynamically allows a retail chain to analyze daily sales trends without delays during peak shopping periods

```
SELECT store_id, product_category, SUM(sales_amount)  
AS total_sales FROM sales_data  
GROUP BY store_id, product_category;
```


- **Finance**

- **Use Case:** Snowflake's *Time Travel* feature helps detect fraud by comparing historical data snapshots

```
SELECT * FROM transactions AT TIME AS OF  
'2024-01-01 00:00:00';
```

- **Healthcare**

- **Use Case:** Snowflake consolidates data from multiple clinics into a single, centralized data warehouse to generate unified patient analytics

```
SELECT clinic_id, AVG(patient_age) AS avg_age  
FROM patient_data GROUP BY clinic_id;
```

1. Real-World Scenarios for Data Warehousing

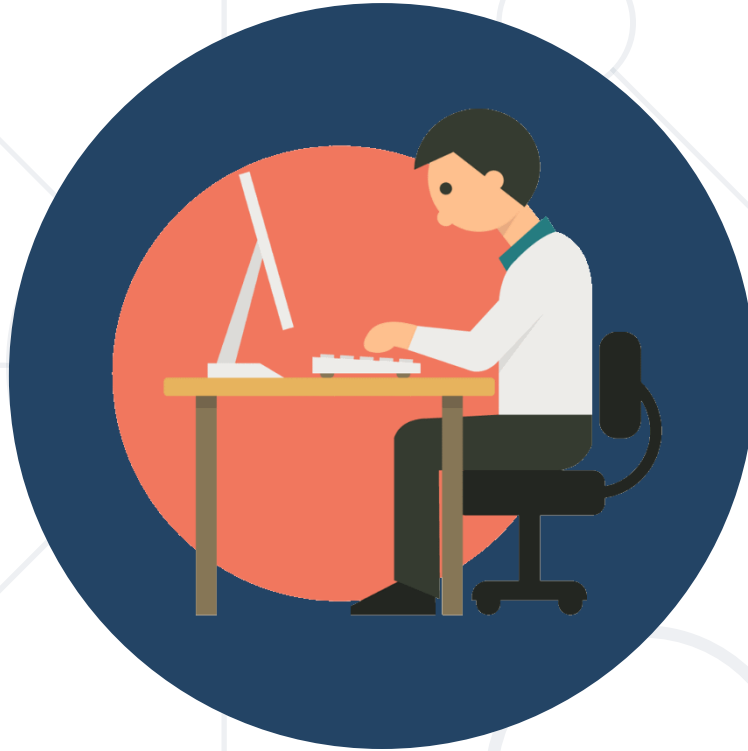
- **Examples:**

- **Amazon**

- *Use Case:* Amazon uses data warehouses to analyze customer purchasing patterns across regions to personalize recommendations and optimize inventory management.
 - *Benefit:* Helps identify high-demand products in specific regions and predict trends.
 - *Scenario:* "Amazon wants to analyze customer behavior during the holiday season. Which features of a data warehouse will make this process efficient?"

■ Netflix

- *Use Case:* Netflix uses a data warehouse to process viewing history and optimize its recommendation engine.
- *Benefit:* Ensures real-time personalization of movie and show recommendations.
- *Scenario:* "Netflix receives millions of data points daily. How does a data warehouse help process this data efficiently?"



Practice

Live Exercise - Snowflake

- Understand the **role** and **architecture** of **data warehousing**
- Learn how **Snowflake** differentiates itself in cloud-native data warehousing
- Learn **Snowflake Architecture, Layers, Pricing** and **Roles**

